Erik Eismann, Marc Stamminger *Erlangen, 03.11.2026*

Compute Rasterization And Tracing For 3D Rendering (exercise sheet 2)

Welcome to the second exercise of CRAT3R - Compute Rasterization And Tracing For 3D Rendering. In this exercise, you will set up a rasterization pipeline, implement texture mapping, and render a relatively complex scene. All the implementation work for this assignment is to be done in `main.cpp`, `simple.vert` and `simple.frag`.

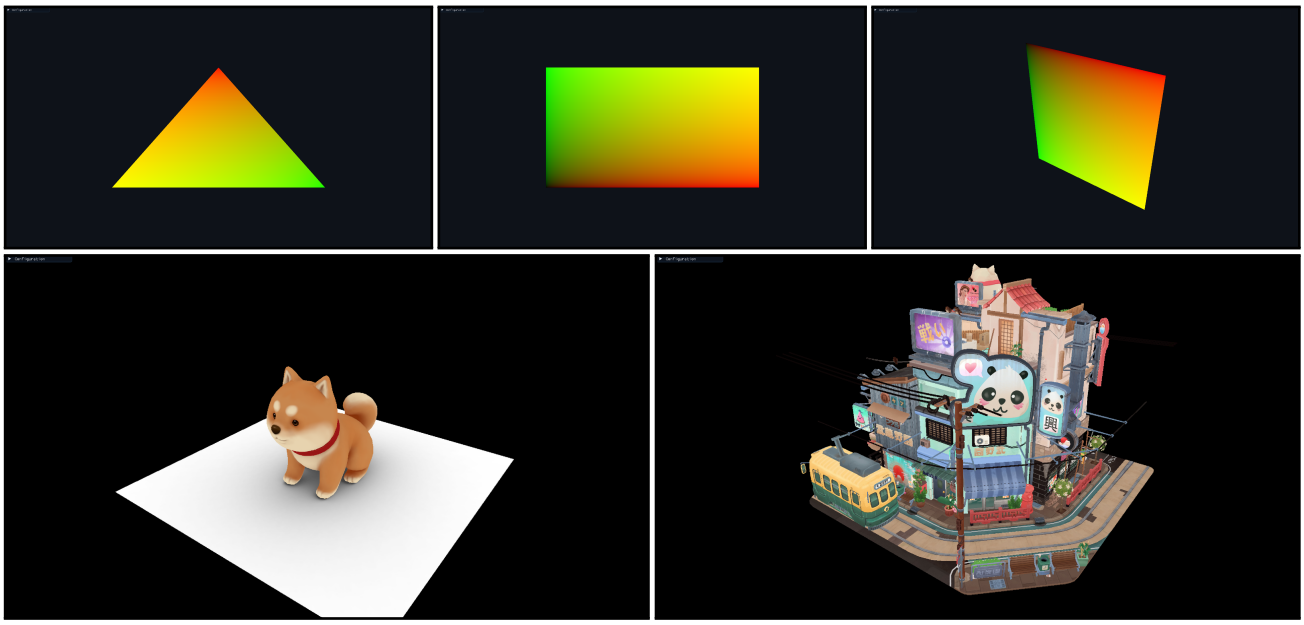


Figure 1: Rendered results of this exercise as you progress through the assignments.

Assignment 2.1 A Simple Triangle

To start with, we will draw just a single triangle to the screen. Take a look at the shaders for this assignment. Currently, these contain the vertices as hardcoded values, so there is no need to upload this data yet. In order to see something, we only need to set up the rasterization pipeline.

Start by implementing `createRenderPass`. This defines which images are attached to the pipeline during rendering and automatically handles layout transitions of the images. In our case, we have a single attachment to which we render the color channels. As we don't care about the previous contents, it starts in the layout `VK_IMAGE_LAYOUT_UNDEFINED`. To render to it, the layout must be transitioned to `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL`. Finally, at the end of the render pass, it must be in `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL` so that the core application can read it and display it on the screen.

Next, configure a very simple pipeline layout in `createPipelineLayout`. Then, it is time to implement `createPipeline`. There are many options to configure here, but as we only want to render a triangle for now, most of them can be disabled. Keep in mind that we are not using any vertex input at this stage, so there is no need to describe the vertex bindings and attributes. Additionally, enable dynamic states for the viewport and scissor.

Afterwards, implement `createFramebuffer` to allocate a framebuffer containing the target image view. When the window is resized, the target image changes. Therefore, the framebuffer must be recreated. The `freeFramebuffer` and `createFramebuffer` functions are already called when this happens. You just have to implement the former function as well.

Finally, implement `recordCommandBuffer` to start rendering something. If you have done everything correctly, you should see a triangle like the one in the top-left image of Figure 1.

Assignment 2.2 Vertex Input

Now, we are going to upload some actual vertex data to the GPU. First, allocate the vertex buffer and upload the data to it in the `createVertexBuffers` function. Next, update `createPipeline` and add the vertex input state to the pipeline. Refer to the `Vertex` struct in `scene.hpp` to see what data defines a vertex. Then, update `simple.vert` accordingly to receive the vertex input. Finally, bind the vertex buffer in `recordCommandBuffer` and update the draw call. Once all these steps have been completed, you should see a quad similar to the one in the top center of Figure 1.

Assignment 2.3 Index Buffer

Although we can now render a quad, we are wasting GPU memory by duplicating vertices. While the quad only has four distinct vertices, we have to upload six to make up the two triangles. Instead, we can use an index buffer to reuse vertex data between triangles. First, create an index buffer in `createIndexBuffers`. Consider what index values the buffer needs to hold. You should then be able to reduce the size of the vertex buffer to only four vertices. Next, bind the index buffer in `recordCommandBuffer` and update the draw call. If implemented correctly, this should result in exactly the same quad.

Assignment 2.4 Push Constants

For now, the vertex shader isn't doing anything. We will use it to transform the vertices using a model-view-projection matrix. In the last exercise, we used a uniform buffer to send constant data to a shader. This would be a valid approach for this exercise too, but we are going to take a different approach. Instead, we will use push constants, which are stored directly in the command buffer. Note that push constants are very restricted in size, with only 128 bytes guaranteed by the Vulkan specification.

Their setup is fairly simple. First, update `createPipelineLayout` to add a push constant range for the vertex shader. Next, upload the push constants to the command buffer before submitting the draw call. Finally, have the vertex shader receive the push constants and apply the MVP matrix to the vertex position. You should then see a quad in 3D space, as in the top-right image of Figure 1, and be able to move around it with your mouse.



Figure 2: Results of the shiba scene of this assignment without and with depth buffering.

Assignment 2.5 Back-Face Culling

Before moving on, now would be a good time to implement back-face culling if you have not done so already. Enable it in the rasterization state when creating the graphics pipeline. If done correctly, the quad should still be visible, but if the camera is rotated around it, it should become invisible from behind.

Assignment 2.6 Triangle Mesh Loading

As a quad is not particularly exciting, we are now going to render real 3D scenes. You will find a loader set up for you in `scene.hpp`. Scenes are saved in the following way: Each scene is made up of a flattened list of nodes. Each node is described by a 4x4 transformation matrix and an index into the list of meshes. A mesh contains vectors of vertices and indices, as well as an index into the material list. Materials have several properties which are not too relevant at the moment, as well as optionally indices into the texture list. Each texture is defined by its width, height, and raw RGBA8 data. Scene loading is also already set up in `main.cpp`. You can specify which scene to load at the top of the file. Change this to `assets/shiba.glb` now.

Next, update `createVertexBuffers` and `createIndexBuffers` to load the scene's mesh. This first scene only contains a single mesh. You can therefore load it directly into your vertex and index buffers. Then, update the number of indices in the draw call and apply the model matrix to the MVP matrix in the push constants. The first scene also only contains a single node that references the only mesh. After implementing all this, you should see a cursed dog as in the left image of Figure 2.

Assignment 2.7 Depth Buffering

The rendered image looks wrong because we are not using depth buffering, which means the order of the triangles is arbitrary. We should fix this now. First, implement `createDepthResources` and `freeDepthResources` to create and free the depth image. Similary to the framebuffer, the depth image has to be recreated when the screen is resized. Then, update `createFramebuffer` to add the depth image view as a second attachment. Next, add a second clear value for the depth image in `recordCommandBuffer`.

We also have to tell our render pass that we are using a depth attachment. Update `createRenderPass` by adding a second attachment description and reference. Finally, add a depth and stencil state to `createPipeline` to enable depth writing and testing during rasterization. Now you should see a correctly rendered UV-dog, as in the right image of Figure 2.



Figure 3: Results of the tokyo scene as you progress through this assignment. Left: with incorrect textures; Center: without alpha blending; Right: without materials that do not use a texture.

Assignment 2.8 Texture Mapping

Now, let's replace those bright UV colors with real colors from a texture. You should be familiar with the steps for this assignment from the first exercise. Implement `createDescriptorSetLayout` and update `createPipelineLayout`. This simple scene only has one texture. Upload it to the GPU in `createTextures`. Refer to the struct in `scene.hpp` to see what makes up a texture in our scene representation. Next, create a sampler in `createTextureSampler`. Thereafter, implement `createDescriptorPool` and `createDescriptorSets`. In this case we do not need seperate descriptor sets for each in-flight frame, as we are using push constants for the variable data. Finally, bind the descriptor set before the draw call and use the texture in the fragment shader (`simple.frag`). If you do everything correctly, you should see a textured dog as in the bottom left image of Figure 1.

Assignment 2.9 Multi-Mesh Scene

We will now switch to a more fully featured scene that features multiple meshes and textures, and a large number of nodes that reuse these meshes. Start by uploading the vertex and index data for all meshes, by updating `createVertexBuffers` and `createIndexBuffers`. Then, update the recording of the command buffer to submit a draw call for each node of the scene. You will have to update the push constants and bind the vertex and index buffers for each node before making the draw call. After implementing this you should see the scene as in the left image of Figure 3. You may notice several errors, such as meshes using wrong textures and missing transparency.

Assignment 2.10 Multiple Materials

Next, we will fix the fact that all meshes are using the same texture, even though the scene uses multiple. First, in `createTextures` load all textures of the scene instead of just the first. Then, reserve enough space in `createDescriptorPool` for a seperate descriptor for each material. Remember, that the number of textures and number of materials is not the same, as multiple materials can use the same texture and some materials may use no texture at all. Subsequently, update `createDescriptorSets` by creating a seperate descriptor set for each material, that references the texture that material uses. Also take care to handle the case of a material that uses no texture. You can compare your results to the center image of Figure 3.

Assignment 2.11 Transparent Objects

You should now notice that several objects, such as bushes and grass, as well as textured details on the walls, have black backgrounds. There are two reasons why this can happen: The first reason is that alpha blending is disabled in the graphics pipeline. The second reason is because some transparent objects are drawn before their background, causing everything behind them to be culled during the depth test. If you only have one layer of transparent objects from every view position and are not too concerned with accuracy, a very simple approach to the second problem is to first draw all opaque objects, then all transparent objects.

Now, let's address these two issues. First, configure alpha blending in `createPipeline` if you have not already done so. Then, update `recordCommandBuffer` so that all opaque objects are drawn first, followed by the transparent objects. Your results should then match the right image of Figure 3.

Assignment 2.12 Single-Color Objects

Finally, we are going to very crudely handle objects that do not use a color texture. These are mainly some of the windows in this scene, which have a semi-transparent constant color. First, update `createDescriptorSetLayout` by adding a binding for a uniform buffer. Next, create and upload one uniform buffer per material. As the material properties do not change, you can use the utility function to upload to a device-local buffer. Then, allocate space for uniform buffer descriptors in the descriptor pool and write the uniform buffers to the descriptor sets. Finally, update the fragment shader to use the constant color in case the material does not have a texture. You should then finally see the scene rendered in its full unshaded glory. You can compare your result to the bottom-right image of Figure 1.

Assignment 2.13 **Curveball** - Order of Transparent Objects

This is the first 'Curveball' assignment. You may find that there are no instructions on how to solve it in the codebase. Instead, you are asked to attempt it without the usual handholdy guidance. Your solutions to the 'Curveballs' do not need to be perfect, but you should be able to explain why you went about it your way.

You may notice that some transparent objects are still not drawn correctly. The blending looks wrong if multiple transparent objects overlap. You should be able to see this if you look through one of the small trees in front of any of the windows. This can be fixed by drawing the transparent objects in the correct order. You are now asked to address this issue. Hint: this can be done with a few lines of CPU code.

Assignment 2.14 **Curveball** - Mip Mapping

Even though we are not really viewing our scenes from very far away, you can still find minification artifacts on textured surfaces, for example if you view them at steep angles. Your task is relatively straight forward: Add mip maps to the textures. Change the necessary parameters of the images, image views, and wherever else necessary. Then, generate the mip levels at runtime by repeatedly blitting to the next level. You might have to adapt `image.transitionLayout` or not use the helper function during mip generation.